Equivalence between models for regex-based information extraction with errors

8 de enero de 2026

Resumen

We formalize and prove the equivalence between two models for regex-based information extraction with errors: (1) a semantics operating on the original document with error annotations, and (2) a semantics working on a modified document with edit operations. We demonstrate a bijective mapping between matches in both models, preserving spans and error operations under consistent index transformations. This results enable interoperability between systems using either approach.

1. Preliminaries

1.1. Basic definitions

Document and spans. A document d is a string over a finit alphabet Σ, we write $d = a_0 a_2 ... a_{n-1}$ to denote a document of legth $|d| = n$. A span over a document d is a pair $s = [i, j\rangle$ of natural numbers i and j with $0 \leq i \leq j \leq |d|$. s is associated with a substring of d from position i to position $j - 1$. We denote this substring by $d(s)$. Given two spans $s_1 = [i_1, j_1\rangle$ and $s_2 = [i_2, j_2\rangle$, the concatenation of s_1 and s_2 is defined as $s_1 \cdot s_2 = [i_1, j_2\rangle$, whenever $j_1 = i_2$.

Mappings. Let V be a fixed set of variables, disjoint from Σ. A *mapping* is a function μ from a finite set of variables $dom(\mu) \subseteq V$ to spans. The empty mapping, denoted by $\varnothing$, is the only mapping such that $dom(\varnothing) = \varnothing$. Similarly, $[x \to s]$ denotes the mapping whose domain only cointains the variable x, which it assign to span s.

Document spanners. Let V be a fixed set of variables and $d \in \Sigma^*$ a document, a (V, d)-tuple is a mapping from V to span(d). A document spanner is a function that map every document d to a set of (V, d)-tuple, such that the range of each $\mu \in M$ are spans of d modeling the process of extracting information from d. For this goal, we will work with 3 models for defining spanners: RGX, ref-words and variable-set automata.

Regex formulas. A *regex-formula* (**RGX** for short) is a regular expression extended with variables in V (called capture variables). Formally, we define the syntax with the recursive rule:

$$e \ := \ \varnothing \mid \epsilon \mid e \mid e \vee e \mid e \cdot e \mid e^* \mid x\{\ e\ \}$$

Spanner defined by a RGX. Let R be a RGX and $d \in \Sigma^*$ a document. The semantics of R is defined as a function $[\![R]\!]$ that assigns to d the set of all mappings μ produced by matching R against d:

$$[\![R]\!](d) := \{\mu \mid \mu \text{ is obtained by a successful match of } R \text{ on } d\}.$$

The semantics of a RGX R is given recursively as follows:

- $[\![\varnothing]\!](d) := \varnothing$.
- $[\![\epsilon]\!](d) := \{\varnothing\}$.
- $[\![R_1 \cdot R_2]\!](d) := \{\mu_1 \cup \mu_2 \mid d = d_1 d_2,\ \mu_1 \in [\![R_1]\!](d_1),\ \mu_2 \in [\![R_2]\!](d_2),\ \mathrm{dom}(\mu_1) \cap \mathrm{dom}(\mu_2) = \varnothing\}$.
- $[\![R_1 \vee R_2]\!](d) := [\![R_1]\!](d) \cup [\![R_2]\!](d)$.
- $[\![R^*]\!](d) := \{\mu_1 \cup \cdots \cup \mu_n \mid d = d_1 \cdots d_n,\ \mu_i \in [\![R]\!](d_i) \text{ para todo } i,\ \mathrm{dom}(\mu_i) \cap \mathrm{dom}(\mu_j) = \varnothing \text{ si } i \neq j\}$.
- $[\![x\{R'\}]\!](d) := \{\mu' \cup [x \mapsto s] \mid s \in \mathrm{span}(d),\ \mu' \in [\![R']\!](d[s])\}$.

Thus, $[\![R]\!](d)$ denotes the set of mappings resulting from all successful matches of R on d.

Variable-set automata. A *variable-set automaton*(VA) can be understood as an ϵ-NFA that is extended with capture variables in a way analogous to RGX; that is, it behaves a usual finite state automaton with $\epsilon-$transitions, except it can also open and close variables. Formally, a VA $\mathcal{A}$ is tuple $(Q, \Sigma, V, q_0, F, \delta)$, where Σ is a finite set of alphabet symbols, V is a finite set of variables, Q is a finite set of states, $q_0 \in Q$ is an initial state, $F \subseteq Q$ is a set of final states and δ is a transition relation consisting of letter transitions of the form (q, a, q') and variable transition of the form $(q, x\vdash, q')$ or $(q, \dashv x, q')$, where $q, q' \in Q$, $a \in Q$ and $x \in V$. We define the set $\mathrm{Vars}(\mathcal{A})$ as the set of all variables that occur in $\mathcal{A}$.

A configuration of a VA $\mathcal{A}$ over a document $d = a_0 a_1 ... a_{n-1}$ is a tuple (q, i), where $q \in Q$ is the current state and $i \leq n$ is the current position in the document. A run ρ of $\mathcal{A}$ is sequence of the form:

$$\rho := (q_0, i_o) \xrightarrow{a_0} (q_1, i_1) \xrightarrow{a_1} \cdots \xrightarrow{a_{n-1}} (q_{n-1}, i_{n-1})$$

where $a_i \in \Sigma \cup \{x\vdash, \dashv x \mid x \in V\}$ and (q_i, a_{i+1}, q_{i+1}). Moreover, $i_0 ... i_{n-1}$ is a non-decreasing sequence such that $i_0 = 1$ and $i_{n-1} = |d|$, and $i_{j+1} = i_j + 1$ if $a_{j+1} \in \Sigma$ and $i_{n-1} = i_j$ otherwise.

Let $\mathcal{A} = (Q, \Sigma, V, q_0, F, \delta)$ be a VA and $d = a_0 a_1 \ldots a_{n-1} \in \Sigma^*$ a document.

The semantics of $\mathcal{A}$ is a spanner $[\![\mathcal{A}]\!]$ that maps d to the set of mappings obtained from all accepting runs of d on $\mathcal{A}$. Each run ρ induces a mapping μ_ρ defined by:

$$\mu_\rho(x) := [i_{x\vdash}, i_{\dashv x})$$

where $i_{x\vdash}$ is the position when the transition $x\vdash$ is taken, and $i_{\dashv x}$ is the position when $\dashv x$ is taken (assuming each $x \in V$ is opened and closed exactly once).

The semantics of $\mathcal{A}$ on d is then given by:

$$[\![\mathcal{A}]\!](d) := \{\mu_\rho \mid \rho \text{ is an accepting run of } \mathcal{A} \text{ on } d\}.$$

Ref-words. For a finite set V of variables, ref-words are defined over the extended alphabet $\Sigma \cup \Gamma_V$, where $\Gamma_v := \{x\vdash, \dashv x \in V\}$, we assume that Γ_V is disjoint with Σ. Ref-words extend strings over Σ by encoding opening $(x\vdash)$ and closing $(\dashv x)$ of variables. A ref-words $r \in (\Sigma \cup \Gamma_V)^*$ is valid if every ocurring variable is opened and closed exactly once. For every valid ref-word r over $(\Sigma \cup \Gamma_V)$ we define $\mathrm{Vars}(r)$ as the set of variables $x \in V$ which ocurr in the ref-word. More formally:

$$\mathrm{Vars}(r) := \{x \in V \mid \exists r_x^{prev}, r_x, r_x^{post} \in (\Sigma \cup \Gamma_V)^*.\ r = r_x^{prev} \cdot x \vdash \cdot r_x \cdot \dashv x \cdot r_x^{post}\}$$

Intuitively, each valid ref-word encondes a mapping for some document d, the variable markers encode where the spans begin and end. Formally, we define functions doc and tup that, given a valid ref-word, output the corresponding document and mapping. The morphism $\mathrm{doc} : (\Sigma \cup \Gamma_V)^* \to \Sigma^*$ is defined as:

$$\mathrm{doc}(a) := \begin{cases} a & \text{if } a \in \Sigma \\ \varepsilon & \text{if } a \in \Gamma_V \end{cases}$$

Now, we have that every valid ref-word has a unique factorization

$$r = r_x^{prev} \cdot x \vdash \cdot r_x \cdot \dashv x \cdot r_x^{post}$$

for each $x \in \mathrm{Vars}(r)$. We can define de function tup as

$$\mathrm{tup}(r) := \{x \to [i_x, j_x\rangle \mid x \in \mathrm{Vars}(r), i_x = |\,\mathrm{doc}(r_x^{prev})|, j_x = i_x + |\,\mathrm{doc}(r_x)|\}$$

The usage of the doc morphism ensures that indices i_x and j_x refer to positions in the document and do not consider other variable operations.

We also define the *trace* morphism $\gamma : (\Sigma \cup \Gamma_V)^* \to \Gamma_V^*$:

$$\gamma(a) := \begin{cases} \varepsilon & \text{if } a \in \Sigma \\ a & \text{if } a \in \Gamma_V \end{cases}$$

1.2. Edit distance

1.2.1. The Edit Distance Problem

In document processing with regular expressions, we often encounter texts with imperfections such as:

- Typos (e.g., documment instead of document)
- Missing characters (e.g., docment)
- Extra insertions (e.g., docuument)
  "a" → "abc". ins(1, b), ins(c,2) — ins (1,c) ins(1,b)

The *edit distance* between two strings D and D', denoted by $\Delta(D, D')$, is the minimum number of elementary edit operations required to transform D into D'. We consider three fundamental operations:

1.2.2. Operations

Given a document $D = a_1 \cdots a_n$:

- **Insertion**: $\mathsf{ins}(p, c)$ - Insert character c at position p

$$D' = a_1 \cdots a_p c a_{p+1} \cdots a_n$$

- **Deletion**: $\mathsf{del}(p, c)$ - Delete character c at position p

$$D' = a_1 \cdots a_{p-1} a_{p+1} \cdots a_n$$

- **Substitution**: $\mathsf{sub}(p, d, c)$ - Replace character d with character c at position p

$$D' = a_1 \cdots a_{p-1} c a_{p+1} \cdots a_n$$

2. Edit Operation Application

2.1. Formal Definition

Given a document $D \in \Sigma^*$ (where $D = d_1 d_2 \cdots d_n$) and a sequence of edit operations $E = [op_1, \ldots, op_m]$, we define the transformed document $E(D)$ recursively as:

$$E(D) = \begin{cases} D & \text{if } E = \emptyset \\ E'(\mathsf{op}_1(D)) & \text{otherwise, with } E' = [op_2, op_3, \ldots, op_m] \end{cases}$$

where each atomic operation op_i modifies the document as follows:

$$\begin{aligned} \mathsf{ins}(p, c)(D) &= d_1 \cdots d_p c d_{p+1} \cdots d_n \quad (0 \leq p \leq n) \\ \mathsf{del}(p, c)(D) &= d_1 \cdots d_{p-1} d_{p+1} \cdots d_n \quad (1 \leq p \leq n) \\ \mathsf{sub}(p, d_p, c)(D) &= d_1 \cdots d_{p-1} c d_{p+1} \cdots d_n \quad (1 \leq p \leq n) \end{aligned}$$

2.2. Properties

For any document D and valid operation sequence E:

1. **Termination**: $E(D)$ always terminates in $O(|E|)$ steps
2. **Length Change**: $|E(D)| = |D| + \#\mathsf{ins} - \#\mathsf{del}$
3. **Order Sensitivity**: $[op_1, op_2](D) \neq [op_2, op_1](D)$ in general

2.2.1. Example

Let $D = \texttt{algorithm}$ and $E = [\mathsf{ins}(3, \texttt{h}), \mathsf{del}(6), \mathsf{sub}(2, \texttt{L})]$. Then:

$$\begin{aligned} E(D) &= [\mathsf{del}(6), \mathsf{sub}(2, \texttt{L})](\mathsf{ins}(3, \texttt{h})(\texttt{algorithm})) \\ &= [\mathsf{del}(6), \mathsf{sub}(2, \texttt{L})](\texttt{alghorithm}) \\ &= [\mathsf{sub}(2, \texttt{L})](\mathsf{del}(6)(\texttt{alghorthm})) \\ &= [\mathsf{sub}(2, \texttt{L})](\texttt{alghothm}) \\ &= \texttt{aLgrithm} \end{aligned}$$

2.3. Application of Edit Sequences on Mappings

Let $\mu : x \to [i, j\rangle$ be a mapping and a edit operations $E = [op_1, \ldots, op_m]$, we define its corresponding transformation on μ, denoted $E(\mu)$, as the mapping obtained by adjusting all affected intervals to maintain positional consistency with the edited document.

Formally, for each operation $\mathsf{op} \in E$:

The extended mapping transformation is defined recursively:

$$E(\mu) = \begin{cases} \mu & \text{if } E = \emptyset, \\ E'(\mathsf{op}_1(\mu)) & \text{otherwise, where } E' = [op_2, \ldots, op_m]. \end{cases}$$

where each atomic operation op_i modifies the mapping as follows:

$$\mathsf{ins}(p,c)(\mu) = \begin{cases} [i+1, j+1\rangle & p \leq i \\ [i, j+1\rangle & i < p \leq j \\ [i, j\rangle & j < p \end{cases}$$

$$\mathsf{del}(p,c)(\mu) = \begin{cases} [i-1, j-1\rangle & p \leq i \\ [i, j-1\rangle & i < p \leq j \\ [i, j\rangle & j < p \end{cases}$$

$$\mathsf{sub}(p,d,c)(\mu) = \mu$$

In other words, each insertion or deletion shifts the intervals of μ that occur after the edit position, ensuring that μ remains consistent with the structure of the transformed document $E(D)$.

This definition guarantees that for any $x \in \mathrm{dom}(\mu)$, the relative alignment between $\mu(x)$ and its corresponding content in $E(D)$ is preserved across arbitrary edit sequences.

2.4. Undo edits

Note that, given a edit sequence E, a document d and the document resulting from applying the edit sequence over d, $E(d) = d'$, we can easily undo de changes taking the inverse of each operation. We will write E^{-1} the inverse sequence of edits and it is defined as follow.

For each atomic operation $op_i \in E$:

$$\mathsf{ins}(p,c)(D)^{-1} = \mathsf{del}(p,c)(D)$$
$$\mathsf{del}(p,c)(D)^{-1} = \mathsf{ins}(p,c)(D)$$
$$\mathsf{sub}(p,d,c)(D)^{-1} = \mathsf{sub}(p,c,d)(D)$$

And finally $E^{-1} = [op_1^{-1}, \ldots, op_n^{-1}]$. Note that this definition works good when we want to undo the edits maded over a mapping μ.

2.5. Normalized edit sequence

Two edit sequences E and E' will we said *equivalent*, $E \equiv E'$ if they generate identical output $E(d) = E'(d)$ when they operate on any document d. Given two documents d and d', there exist infinte ways to transform d in d'. In order to compare edit sequences, it is convenient to define a normalized form. For this purpose, supose that in a edit sequence $E = [op_1, \ldots, op_n]$, the atomic operation op_i work in the position $d(i)$, we will write $op_i \prec op_j$ if either $i < j$

3. Models for information extraction with errors

Now, we will consider two diferent approaches to aboard the edit distance problem in Rematch engine.

3.1. Error over a modified document

The first option is the straightforward version. Take a document d, compute all the possibles documents d' that the edit distance between d and d' is less than k and prove if d' matches de REQL expresion.

This is the semantics for this approach:

Given a document $d \in \Sigma^*$ (where $d = d_1 d_2 \cdots d_n$) and a REQL expression R, we define the semantics of R with k error as it follows:

$$[\![R^{\leq k}]\!]_d = \{(\mu, E) \mid \exists d'.E(d) = d' \wedge \mu \in [\![R]\!]_{d'} \wedge |E| \leq k\}$$

We make a match in a new document d' that is the result of apply the edits in E over d. The span and the mapping is over d' because we only compute d' and treat it like the original document.

3.1.1. Example

Using the same example as before, with $d = a$, $e = a!x\{b\}c$ and $k = 2$ the following is a valid match in the new semantics:

$$([0, 3\rangle, \mu := [x \mapsto [1, 2\rangle], E = (ins(1, b), ins(2, c))$$

The document d' that make the match is $d' = abc$, the span and mapping is over this document.

Now we give the user the new document and the list of edits that we make over the original to get the new one.

3.2. Error over the original document

Given a document $d \in \Sigma^*$ (where $d = d_1 d_2 \cdots d_n$), a REQL expression R, and an edit sequence $E = [op_1, \ldots, op_m]$, we define the semantics of R with k errors over the *original* document as:

$$[\![R + k]\!]_d = \{\, (\mu, E) \mid E(\mu) \in [\![R]\!]_{E(d)} \wedge |E| \leq k \,\}$$

where μ is a mapping over the original document d, and $E(\mu)$ denotes the mapping obtained by applying the same edit sequence E to the spans of μ, ensuring positional consistency with the transformed document $E(d)$.

In this formulation, the match is reported over the original document d, while E represents the minimal sequence of edits required to make d satisfy the REQL expression R.

4. Canonical form for edit sequences

We define a canonical (normal) form for edit scripts so that equivalent scripts (those producing the same output on any input and inducing the same span transformations) admit a unique representative. This is crucial both for comparison and for interoperability with extraction semantics.

4.1. Equivalence

Two edit sequences E and E' are *equivalent*, written $E \equiv E'$, if for every document D we have $E(D) = E'(D)$ and, for every mapping μ over D, the induced mappings coincide: $E(\mu) = E'(\mu)$ (as defined in Section 2).

4.2. Anchors and left-to-right discipline

Let $E = [op_1, \ldots, op_m]$ be an edit sequence applied to D from left to right. For each t, let $D^{(t)}$ be the document after applying $op_1, \ldots, op_{t-1}$. We call the *anchor* of op_t, denoted $\alpha(op_t)$, the index p mentioned in op_t but interpreted over the *current* string $D^{(t)}$ (as in our operational semantics). Intuitively, α says "where in the current cursor the edit occurs".

A script is *left-to-right* if the sequence of anchors never goes backwards after transporting them to the original coordinate system. Formally, define as follows: for $s < t$,

$op_s op_t$ iff when replaying $op_1, \ldots, op_{t-1}$ on D, $\alpha(op_s)$ refers to a position no greater than the transported $\alpha(op_t)$.

Our canonical scripts will satisfy that the resulting order of operations is compatible with (no "jumping back" to earlier prefixes).

4.3. Local rewrite system

We define a terminating set of local rewrites that preserves equivalence ($\equiv$). They act on adjacent operations whose anchors (in the current string) coincide at transformation time.

Cancellation and fusion at the same anchor. For any character $a, c \in \Sigma$:

$$\begin{aligned} \mathrm{ins}(p,c);\ \mathrm{del}(p,c) &\Rightarrow \varepsilon && \text{(insertion immediately deleted)} \\ \mathrm{del}(p,a);\ \mathrm{ins}(p,c) &\Rightarrow \mathrm{sub}(p,a,c) && \text{(delete+insert = substitute)} \\ \mathrm{sub}(p,a,b);\ \mathrm{sub}(p,b,c) &\Rightarrow \mathrm{sub}(p,a,c) && \text{(collapse adjacent substitutions)} \end{aligned}$$

Precedence at the same anchor. When two adjacent operations $op_1; op_2$ share the anchor and no rule above applies, we impose the following order (move left if needed):

$$\mathrm{del} \prec \mathrm{sub} \prec \mathrm{ins}.$$

In words: do all deletions *first*, then at most one substitution, then insertions. This matches the intuition "consume input before producing output".

Commutation at different anchors. If two consecutive operations act on disjoint anchors and swapping them does not change the transported anchors nor the result, we commute them to enforce the global left-to-right discipline (earlier anchor first). Concretely, whenever applying $op_1; op_2$ yields the same D' and the same mapping action as $op_2; op_1$ (after index transport), we rewrite to the order in which the transported anchor is smaller.

4.4. Canonical (normal) form

A script E is in *canonical form* if:

1. (**Left-to-right**) Its operation order is compatible with the transported anchor order (no backward jumps to earlier prefixes).
2. (**Local normality**) No local rule above applies: there are no cancellable ins; del pairs, no adjacent substitutions to merge, and the precedence at each anchor is $\mathrm{del}^*; \mathrm{sub}^{\leq 1}; \mathrm{ins}^*$.

Theorem 1 (Termination)**.** *The rewrite system above terminates.*

Proof sketch. Use a multivariate measure that strictly decreases on each rewrite: (i) the number of precedence violations at equal anchors, (ii) the number of cancellable or fusible adjacent pairs, (iii) the number of anchor order inversions after transport, (iv) the total length of the script. Each rule reduces the lexicographic tuple. □

Theorem 2 (Confluence up to $\equiv$)**.** *Any two rewrite sequences from the same script yield $\equiv$-equivalent normal forms.*

Proof sketch. Overlaps occur only at length-2 windows around the same anchor; all critical pairs are joinable because all outcomes produce the same character at that anchor and the same span-transport effect (by construction of the rules). For disjoint anchors, commutation preserves both $E(D)$ and $E(\mu)$. □

Theorem 3 (Uniqueness of canonical form)**.** *Every script E has a unique canonical representative* $\mathrm{NF}(E)$ *modulo $\equiv$.*

4.5. DP-based construction with deterministic tie-breaking

In practice, we build canonical scripts from the Wagner–Fischer DP table. Let D and D' be the input and target strings, and let moves be: diagonal = match/substitution, up = deletion, left = insertion. Among all optimal paths, choose the lexicographically smallest under

$$\texttt{diag} \prec \texttt{up} \prec \texttt{left},$$

breaking ties by preferring to move *towards smaller input indices first* (i.e., at a tie between `up` and `left`, pick `up`). This ensures that, at each input position, we first delete, then possibly substitute, and finally insert—matching our local precedence.

We then emit the operation stream by scanning the chosen path forward:

- `diag` with equal symbols $\Rightarrow$ no-op;
- `diag` with different symbols $\Rightarrow$ $\mathrm{sub}(p, a, c)$;
- `up` $\Rightarrow$ $\mathrm{del}(p, a)$ (consume a at p);
- `left` $\Rightarrow$ $\mathrm{ins}(p, c)$ (produce c at p).

Finally, run the local rewrites to saturation (linear time in the script length due to the local nature of rules). The result is in canonical form by construction.

Complexity. DP takes $O(|D| \cdot |D'|)$ time; the emitted script has length at most that distance. Normalization is $O(|E|)$.

4.6. Interaction with span transport

The precedence del $\prec$ sub $\prec$ ins preserves a clean "consume then produce" discipline: insertions do not shift yet-unprocessed spans to the left, and deletions shift only what lies strictly to their right. Therefore, the mapping-transport rules in Section 2 coincide across all equivalent scripts and are preserved by normalization. In particular, if (μ, E) is a match under the original-document semantics, then $(\mu, \mathrm{NF}(E))$ is an equivalent match with a canonical script.

4.6.1. Example

Given the document $d = a$ and the regex $e = a!x\{b\}c$, with $k = 2$ two errors available, the following is a match in this sematics.

$$\Big([0, 1\rangle,\ \mu := \{x \mapsto [1, 1\rangle\},\ E = [\mathsf{ins}(1, \mathsf{b}), \mathsf{ins}(1, \mathsf{c})]\Big)$$

4.6.2. Idea para computar

Usar un automata de levenstein que lleve registro de las posiciones del documento donde se hicieron los errores.

5. Equivalence Proof

Theorem 4. *For every document $d \in \Sigma^*$, REQL R, and budget $k \in \mathbb{N}$, there is a bijection between*

$$[\![R^{\leq k}]\!]_d \quad \textit{and} \quad [\![R + k]\!]_d,$$

given by translating mappings via the action of the edit sequence E on spans.

Demostración. We define two inverse functions between the sets and prove they are well-defined and mutually inverse.

5.0.1. $[\![R^{\leq k}]\!] \implies [\![R+k]\!]_d$

Given $(\mu', E) \in [\![R^{\leq k}]\!]_d$, then there exists d' with $E(d) = d'$ and $\mu' \in [\![R]\!]_{d'}$ and $|E| \leq k$. Let $\mu := E^{-1}(\mu')$. Then $E(\mu) = E(E^{-1}(\mu')) = \mu'$, hence

$$E(\mu) \in [\![R]\!]_{E(d)}.$$

By definition of the original-document semantics, $(\mu, E) \in [\![R+k]\!]_d$ with $|E| \leq k$.

5.0.2. $[\![R+k]\!]_d \implies [\![R^{\leq k}]\!]_d$

Given $(\mu, E) \in [\![R+k]\!]_d$, then $E(\mu) \in [\![R]\!]_{E(d)}$ and $|E| \leq k$. Let $d' = E(d)$. Then $(E(\mu), E) \in [\![R^{\leq k}]\!]_d$ by the modified-document semantics.

□